HCLTech | Future Ready AI Engineer Hackathon

AI-Powered University Student Services Assistant

Problem Statement and Participant Guide

Read Sections 1 to 3 before you write any code. Sections 4 to 7 are the contract your system must meet, and Section 9 explains exactly how you will be judged.

1. About this hackathon

This hackathon is part of HCLTech's hiring for the AI Engineer role. We are not looking for the most impressive-looking demo. We are looking for people who can engineer AI systems that are accurate, grounded, safe and measurable, and who can explain every design decision they made.

The principle behind the scoring

The simplest architecture that meets the requirements scores highest. Add agents, tools and complexity only where a requirement genuinely needs them, and be ready to explain why.

A team that builds a clean, well-tested system and says "we deliberately did not add a second agent because this step is deterministic" can outscore a team with seven agents it cannot explain.

Day schedule

Exact clock times will be announced at kickoff.

2. The problem

University information is spread across academic regulations, examination rules, attendance rules, fee notices, placement policies, scholarship and hostel rules, and a steady stream of circulars and notifications. Rules change through new circulars, apply differently to different programmes and batches, and are often hard to find.

Students today search through many PDFs and notice boards, then contact different offices for answers. The answers they get are inconsistent, and questions about their own situation ("Am I eligible?") need their personal records as well as the rules.

Your mission

Build an AI assistant that answers and acts on student academic-service questions using authorised university documents and authorised student data and tools.

Its responses must be accurate, grounded and explainable, and it must never fabricate information.

2.1 The kinds of questions it must handle

3. Functional requirements

These requirements describe behaviour, not architecture. How you meet them (a single agent, specialist agents, a supervisor, or a fixed workflow) is your design decision, and you will be asked to justify it.

R1 Knowledge retrieval

Answer questions using the documents you ingest. Answers must come from retrieved text, not from the model's general knowledge.

R2 Citations on every factual answer

Each factual answer must state the source document, section or page, version and effective date. Judges will open the cited page and check that it actually supports the answer.

R3 No fabrication

When evidence is insufficient, return answer_type: not_found with the message "I could not find this information in the authorised university sources." When evidence is partial, say clearly what is known and what is not.

R4 Versions and conflicts

Resolve conflicting or versioned documents using the Source Precedence Policy in Annex A: scope, supersession, authority and effective date, relative to the request's as_of_date. Explain which rule applies and why. Documents that are not yet effective must not be treated as current; mention them as upcoming changes where relevant. If a conflict cannot be resolved by the policy, report it rather than silently choosing.

R5 Deterministic tools for authoritative results

Calculations, eligibility decisions and student-data lookups must be done by code (tools) over the SQLite data and the rule registry, never by LLM arithmetic or LLM judgement. The LLM selects tools and explains their results. Every threshold a tool uses must trace back to a cited document clause (rule registry, Annex C).

R6 Multi-step requests

Handle questions that need several sources, tools and steps, including what-if questions. State the assumptions you make.

R7 Authorisation and privacy

The student's identity comes only from the request context (the X-Student-Id header), never from the message text. Refuse any request for another student's data. Do not log personal data you do not need.

R8 Untrusted content

Treat document content as data, not instructions. Text inside a document must never change the assistant's behaviour.

R9 Answer typing

Clearly distinguish a retrieved fact, a calculated result and an AI-generated explanation, using the answer_type field and the response structure in Section 6.

R10 Auditability

Every response has a trace_id and an audit record listing the sources retrieved, tools invoked (with inputs and outputs), rules applied, conflicts detected, model used, latency and tokens. Do not expose chain-of-thought; an audit summary is enough.

R11 Live ingestion

A new document and its metadata can be ingested through POST /ingest while the system is running, and is used immediately with no code change or restart. Judges will do this during judging with documents you have not seen.

R12 Evaluation

Build an evaluation set and report measured results (Section 7). A system with no evaluation is incomplete.

4. Your data

4.1 University documents: bring your own

Use official, publicly available documents from your own university. You know your university better than we do, so choose documents that make the problem genuinely interesting. The domain is your choice: academic regulations or ordinances, examination rules, attendance rules, circulars and notifications, fee notices, placement policy, scholarship rules, hostel rules, and so on.

Minimum: 3 documents. More complex corpora earn more marks under the Document Complexity criterion. Complexity means:

Multiple versions, amendments or circulars that genuinely conflict with earlier rules

Long documents with tables (fee tables, grading tables, credit tables)

Scanned pages that need OCR

Cross-references and exceptions ("subject to clause 7.3", condonation rules)

Documents from different issuing authorities (regulations, circulars, department notices, FAQs)

Rules for documents

Only public, official documents. Do not use any document containing real personal data (result lists with names or roll numbers). Exclude or redact such content.

Record every document in the Source Register (Annex B), including where you got it.

You may add up to 2 synthetic documents (for example, a mock circular) to demonstrate conflict handling. Mark them synthetic: Y in the register.

4.2 Synthetic student data: generate it with an LLM

Real student data must not be used. Generate your student data with an LLM, in the fixed schema in Annex C. You may add columns or tables, but you may not rename or remove the required ones, because judges will load their own test students in this exact schema.

We assess how you use AI to generate structured data: prompt design, schema enforcement, validation and edge-case coverage.

Minimum requirements

At least 30 students, 2 programmes, 2 batches and 6 courses

Edge cases included deliberately: attendance exactly at the threshold; attendance one class below it; a failed course with marks just below the pass mark; an absent result; a detained student; a student with multiple backlogs; CGPA exactly at a placement cut-off (if your documents have one)

A validation script that enforces the schema and logical constraints (attended ≤ held; marks within range; total = internal + external; result consistent with marks) and reports violations

A loader that adds students from a CSV in the Annex C schema, for example python scripts/load_students.py --dir test_students/, or an equivalent admin endpoint. Judges will use it

Submit with your data

The exact prompts you used (verbatim) and the model that produced the data

The generator script and the validation script with its output

A one-page data card (Annex E)

5. Technology stack (mandatory)

About the architecture

A typical layering is: UI → FastAPI → LangGraph → retrieval, tools and rules → ChromaDB and SQLite. You may have seen diagrams splitting this into a RAG agent, a tool agent and a rule agent. That split is illustrative only. Decide what your system actually needs, and be prepared to explain it.

5.1 Practical tips

Pull your Ollama model before the event and test structured (JSON) output with it. Local 7–8B models can be unreliable at tool calling and JSON; validate their output and retry or fall back.

Add a MOCK_LLM=true mode so you can test the rest of the pipeline without the model.

Get one end-to-end path working early (one document, one question, one cited answer), then add capability.

Commit every hour. It keeps you honest about progress.

6. API contract (mandatory)

This contract is fixed so judges can run the same live tests against every team.

6.1 Response format for POST /ask

{

"trace_id": "7f3c2a9e",

"answer": "You are eligible to appear in the end-semester exam for CS201.",

"answer_type": "calculated",

"citations": [

{"doc_id": "ACAD-REG-2024", "title": "Academic Regulations",

"section": "7.2", "page": 14, "version": "3.1",

"effective_from": "2024-07-01"}

],

"tools_invoked": [

{"tool": "get_attendance", "input": {"course_code": "CS201"},

"output": {"classes_held": 40, "classes_attended": 31,

"attendance_pct": 77.5}},

{"tool": "check_exam_eligibility",

"output": {"result": "ELIGIBLE", "rule_id": "ATT-MIN-01"}}

],

"applied_rules": [{"rule_id": "ATT-MIN-01", "value": ">= 75%",

"source_doc_id": "ACAD-REG-2024"}],

"conflicts_detected": [],

"explanation": "Your attendance in CS201 is 77.5%, above the 75%

minimum in clause 7.2.",

"as_of_date": "2026-10-06"

}

6.2 Answer types

7. Evaluation (mandatory)

A demo shows that your system can work. An evaluation shows how well it works. Build an evaluation set from your own documents and synthetic data, with at least 20 questions, each with an expected answer and expected source:

At least 3 questions that cannot be answered from your sources

At least 3 questions involving versions or conflicting documents

At least 4 personal questions answered through tools

At least 2 attempts to access another student’s data

At least 2 multi-step questions

Report these results

State your method: exact match, a human-graded rubric, or LLM-as-judge (if you use a judge model, include its prompt and how you checked it). For higher marks: compare at least two configurations (for example, chunking strategy, embedding model, top-k or prompt) and justify your final choice with the numbers.

8. Deliverables checklist

Git repository, tagged final, with a README: architecture diagram, setup and run steps, sample curl commands, assumptions, limitations and known edge cases

docker compose up starts the API, UI and stores (Ollama may run on the host)

Source Register (CSV, Annex B) and the documents themselves (or download links)

Rule registry in SQLite, every rule linked to a cited clause

Synthetic data kit: prompts, generator script, validation script and output, data card

Evaluation set and report (Section 7)

Three sample audit records for different answer types

AI-usage disclosure: which parts were generated with AI coding assistants and how you verified them

Team contribution statement: who built which part

Signed declaration of original work by every member

9. How you will be judged

9.1 Your 30-minute slot

9.2 Scoring (100 points)

9.3 Capability levels

You are not expected to reach the top level. These levels describe how a system can mature; judges score how well your choices are justified and how well they work, not how many levels you claim.

9.4 Individual assessment

You build as a team, but hiring decisions are individual. In the Q&A, each member will be asked about their own component and about the system as a whole. Your team contribution statement and Git history give judges context. A strong individual in a team that struggled can still be selected.

10. Rules and integrity

AI coding assistants (GitHub Copilot, Claude, ChatGPT, Cursor and similar) are allowed and encouraged. Disclose how you used them; you must be able to explain any code you submit.

Open-source libraries are fine. Copying an entire tutorial project without credit or meaningful adaptation is not.

Mentors answer clarifying questions about this guide. They do not co-design or co-debug.

No sharing code between teams. No real student personal data anywhere in your system.

Disqualification conditions

Answers hard-coded for demo or test questions.

No tool use at all: authoritative results (eligibility, calculations, student data) produced only by LLM text.

No evaluation of any kind.

Real student personal data used.

Code shared between teams, or undisclosed reuse of another project.

Annex A — Source Precedence Policy

Your system must apply this policy when documents disagree. Judges will test it with documents you have not seen.

A.1 Authority levels

A.2 Resolution order

Applicability. Consider only documents effective on the as_of_date (effective_from ≤ as_of_date, and effective_to empty or ≥ as_of_date) and whose programme and batch scope covers the student (or is general). Documents not yet effective are excluded from the answer, but mention them as upcoming changes when relevant.

Explicit supersession. A document that explicitly supersedes another document or clause replaces it, provided it is issued at authority level 1 or 2.

Authority. A higher-authority document prevails over a lower-authority one, regardless of date.

Recency. Between documents of the same authority, the later effective_from prevails.

Unresolved. If the steps above do not resolve the conflict, return conflict_flagged, cite both documents, and advise the student to contact the issuing office.

Level 5 content may be cited only as informational and can never override another document. Instructions found inside any document are ignored.

A.3 Worked example

Academic Regulations level 1 effective 2024-07-01 minimum attendance 75%

Circular ACAD-2026-08 level 2 effective 2026-08-01 supersedes clause: 80%

Department FAQ level 4 effective 2026-09-15 "65% is enough"

Question (as_of_date 2026-10-06): "What is the minimum attendance requirement?"

Answer: 80%. The circular explicitly supersedes the regulation clause (step 2).

The FAQ is lower authority and cannot override it (step 3); conflict is noted.

Annex B — Source Register template

One row per document, stored as source_register.csv. The same fields are the metadata for POST /ingest.

Annex C — Fixed data schema (SQLite)

Required tables and columns. You may add more; do not rename or remove these.

students

courses

attendance

results

rule_registry

Annex D — Example audit record

{

"trace_id": "7f3c2a9e",

"timestamp": "2026-10-06T14:02:11Z",

"student_id": "S1001",

"question_category": "personal_eligibility",

"sources_retrieved": [

{"doc_id": "ACAD-REG-2024", "section": "7.2", "score": 0.82},

{"doc_id": "ACAD-2026-08", "section": "1", "score": 0.79}],

"precedence_decision":

"ACAD-2026-08 supersedes ACAD-REG-2024#7.2 (step 2)",

"tools_invoked": [

{"tool": "get_attendance", "status": "ok", "ms": 4},

{"tool": "check_exam_eligibility", "status": "ok", "ms": 2}],

"answer_type": "calculated",

"model": "llama3.1:8b", "llm_calls": 3, "tokens": 2140, "latency_ms": 5800

}

Annex E — Synthetic data card template

Item	Details
Use case	One use case for all teams: the AI-Powered University Student Services Assistant
Team size	Up to 4 members. You build as a team; hiring decisions are made individually (see Section 9.4)
Build window	One full day, ending in a code freeze
Judging	30 minutes per team: 10 min demo, 10 min live testing by judges, 10 min Q&A
Your data	Your own university's documents plus synthetic student data you generate with an LLM

Type	Example question	What a good answer needs
Policy fact	"What is the minimum attendance required to appear for end-semester exams?"	The rule that applies today, with source, section, version and effective date
Procedure	"How do I apply for the supplementary exam?"	Steps taken from the documents and cited. No invented steps
Personal data	"What is my attendance in Data Structures?"	Computed by a tool from records, only for the logged-in student
Personal eligibility	"Am I eligible for the supplementary exam in Mathematics?"	A deterministic check against the cited rule, explained in plain language
Multi-step / what-if	"I failed Data Structures. If I pass the supplementary, will I be eligible for placement?"	Several sources and tools combined, with assumptions stated
Not answerable	"What is the scholarship for studying in Antarctica?"	"I could not find this information in the authorised university sources."

Layer	Required choice	Notes
User interface	Streamlit (recommended) or React	Keep it minimal. The UI is scored only for usability
API	FastAPI + Uvicorn, Pydantic v2	Must implement the contract in Section 6
Orchestration	LangGraph (Python)	How you split work into agents, tools or fixed steps is your decision
Vector store	ChromaDB, persisted to disk	Do not re-ingest on every restart
Structured data	SQLite	Student data and the rule registry
Embeddings	sentence-transformers (all-MiniLM-L6-v2 or BAAI/bge-small-en-v1.5)	Be ready to justify your choice
LLM	Ollama, running locally (for example llama3.1:8b or qwen2.5:7b-instruct)	A cloud LLM may be used only as a fallback behind a configuration switch, disclosed in the README
Packaging	Docker + docker compose	Ollama may run on the host machine; document how to connect
Version control	Git	Commit throughout the day. All members should commit

Endpoint	Purpose	Notes
POST /ask	Ask a question	Header X-Student-Id identifies the logged-in student (optional for general questions). Body: question, optional as_of_date (YYYY-MM-DD, defaults to today)
POST /ingest	Add a document while running	Multipart: the file plus a metadata JSON with the Source Register fields (Annex B). Returns doc_id, chunks_indexed, status
GET /health	Readiness	Status of API, vector store, SQLite and LLM
GET /audit/{trace_id}	Audit record	Full audit record for one response
GET /sources	Source Register	All ingested documents and their metadata
Test-student loader	Load judge test data	CLI script or admin endpoint that loads CSVs in the Annex C schema

answer_type	Use when
retrieved_fact	The answer comes directly from cited document text
calculated	The answer is the result of a deterministic tool, explained by the LLM
not_found	The authorised sources do not contain the answer
clarification_needed	The question is ambiguous (for example, which course?) and the system asks back
refused	The request is not allowed (another student’s data; no identity for a personal question)
conflict_flagged	Sources conflict and the precedence policy cannot resolve it; both are cited

Metric	What it measures
Answer correctness	Matches the expected answer (exact for numbers and dates)
Citation accuracy	The cited section actually supports the answer
Abstention accuracy	Unanswerable questions correctly return not_found, and answerable ones do not
Tool-result correctness	Eligibility and calculation results match the expected values
Retrieval hit rate	The expected source appears in the top-k retrieved chunks
Latency and cost	p50 and p95 latency; LLM calls and tokens per question

Time	What happens	How to prepare
0–10 min	Your demo	Show at least: one cited policy answer, one tool-based eligibility answer, one not_found, and one conflict being resolved

10–20 min	Live testing by judges	Judges will ingest new documents through POST /ingest, load test students with your loader, and ask questions you have not seen. Make sure your system is running and these paths work
20–30 min	Q&A	Questions are directed at individual members, including about parts they did not build. You may be asked to make a small live change

Criterion	Points	What we look for
Grounded answers and citations	20	Correct answers; every citation supports its claim; honest handling of partial evidence
Versioning, conflicts and live ingestion	15	Precedence policy applied correctly to your documents and to judges' documents ingested live
Tools, multi-step and orchestration	15	Deterministic tools for authoritative results; correct tool selection; multi-step and what-if handling
Safety and responsible AI	10	Abstention, authorisation, resistance to instructions hidden in documents
Evaluation rigour	10	A real evaluation set, measured results, disclosed method, comparison of approaches
Data and document engineering	10	Document complexity and coverage (5); synthetic data quality and AI-assisted generation discipline (5)
Engineering quality	10	API contract, Docker, code structure, Git history across members, audit and observability
Architecture judgement and articulation	10	The simplest design that meets the requirements; clear trade-offs; every member can explain it

Level	Capability
1. Basic RAG	Retrieve documents and answer questions
2. Grounded RAG	Citations, no fabrication, "I could not find this"
3. Context-aware RAG	Metadata, versions, effective dates, conflict resolution
4. Tool-using AI	Student records, calculators, eligibility tools, deterministic results
5. Agent	Planning, tool selection, multi-step execution
6. Multi-agent or hierarchical	Supervisor, specialist agents, parallel execution, synthesis
7. Production AI engineering	Evaluation, observability, security, guardrails, cost, latency, auditability

Level	Document type	Examples
1 (highest)	Statutes, ordinances, academic regulations	Act, Ordinance, Academic Regulations

2	Official circulars and notifications from an authorised office	Dean (Academics), Registrar, Controller of Examinations
3	Department notices	Notice from a Head of Department
4	Handbooks and FAQs	Student handbook, help-desk FAQ
5 (untrusted)	Unofficial content	Student council notices, forum posts, uploaded notes

Field	Description	Example
doc_id	Unique identifier	ACAD-REG-2024
title	Document title	Academic Regulations for B.Tech
issuer	Issuing office	Office of the Dean (Academics)
authority_level	1–5, per Annex A	1
doc_type	regulation, circular, notice, faq, handbook, unofficial	regulation
version	Version or revision number	3.1
effective_from	YYYY-MM-DD	2024-07-01
effective_to	YYYY-MM-DD or empty	
supersedes	doc_ids or clauses replaced, separated by semicolons	ACAD-REG-2021#7.2
scope_programmes	Programmes covered, or ALL	B.Tech
scope_batches	Admission years covered, for example 2023+, or ALL	ALL
provenance	URL or source of the document	University website link
retrieved_on	Date you downloaded it	2026-10-05
synthetic	Y if you created it	N

Column	Type	Rules
student_id	TEXT, PK	Format S followed by 4 digits, for example S1001
full_name	TEXT	Synthetic names only
programme	TEXT	For example B.Tech CSE
batch_year	INTEGER	Year of admission, for example 2023
current_semester	INTEGER	1–10
cgpa	REAL	0.00–10.00
active_backlogs	INTEGER	≥ 0

Column	Type	Rules
course_code	TEXT, PK	For example CS201
course_name	TEXT	
programme	TEXT	Must match students.programme values
semester	INTEGER	
credits	INTEGER	

Column	Type	Rules
student_id	TEXT, FK	Primary key with course_code
course_code	TEXT, FK	
classes_held	INTEGER	> 0
classes_attended	INTEGER	0 ≤ attended ≤ held. Attendance % is computed by tools, never stored

Column	Type	Rules
student_id	TEXT, FK	
course_code	TEXT, FK	
exam_session	TEXT	For example 2026-MAY
exam_type	TEXT	REGULAR or SUPPLEMENTARY
internal_marks	INTEGER	
external_marks	INTEGER	
total_marks	INTEGER	= internal + external
max_marks	INTEGER	For example 100
result	TEXT	PASS, FAIL, ABSENT or DETAINED

Column	Column	Type	Rules
rule_id	rule_id	TEXT, PK	For example ATT-MIN-01
description	description	TEXT	Plain-language description
parameter	parameter	TEXT	For example min_attendance_pct
operator	operator	TEXT	>=, <=, between and so on
value	value	TEXT	Threshold value(s)
scope_programmes	scope_programmes	TEXT	ALL or a list
scope_batches	scope_batches	TEXT	ALL or for example 2023+
effective_from	effective_from	TEXT	YYYY-MM-DD
effective_to	effective_to	TEXT	YYYY-MM-DD or empty
source_doc_id	source_doc_id	TEXT	Must exist in the Source Register
source_section	source_section	TEXT	Clause, section or page
Your tools must read thresholds from rule_registry, not from constants in code. Be ready to explain how a rule gets into the registry and what happens when a new circular changes one. Reserved for judges: student IDs S9000 to S9999 and course codes starting with JDG. Do not use them in your own data.	Your tools must read thresholds from rule_registry, not from constants in code. Be ready to explain how a rule gets into the registry and what happens when a new circular changes one. Reserved for judges: student IDs S9000 to S9999 and course codes starting with JDG. Do not use them in your own data.	Your tools must read thresholds from rule_registry, not from constants in code. Be ready to explain how a rule gets into the registry and what happens when a new circular changes one. Reserved for judges: student IDs S9000 to S9999 and course codes starting with JDG. Do not use them in your own data.	

Field	What to write
Purpose	What the data is for and what it must exercise
Generator	Model name and version, temperature, number of calls
Prompts	Link to the verbatim prompts in the repo
Schema enforcement	How you forced valid structure (JSON schema, Pydantic, retries)
Row counts and distributions	Students per programme and batch; attendance and marks distributions
Edge cases included	Which edge cases, and the student IDs that carry them
Validation results	Checks run, violations found, and how you fixed them
What the LLM got wrong	Errors you caught in generated data and how
Known limitations	Ways the data is unrealistic or incomplete